

# RTL Design Sherpa

# APB PM/ACPI Micro-Architecture Specification 1.0

## January 4, 2026

# Table of Contents

1 Pm Acpi Mas Index....8

2 pm_acpi – Overview....9

2.1 Overview....9

2.1.1 Figure 1.1: PM/ACPI Block Diagram....10

2.2 Parameters....10

2.2.1 Input Synchronizers....11

2.3 Functional Description....11

2.3.1 Register Summary....11

2.3.2 Sticky Status and the Interrupt....14

2.3.3 Power State FSM....14

2.4 Waveforms....15

2.4.1 Waveform 1.1: Sleep Entry (S3 Suspend)....15

2.4.2 Waveform 1.2: Wake Event....15

2.4.3 Waveform 1.3: PM Timer....16

2.4.4 Waveform 1.4: General Purpose Event (GPE)....16

2.5 Navigation....17

3 pm_acpi – Register Map....18

3.1 Overview....18

3.1.1 Register Summary....18

3.2 Functional Description....21

3.2.1 ACPI_CONTROL (0x000)....22

3.2.2 ACPI_STATUS (0x004)....23

3.2.3 ACPI_INT_ENABLE (0x008)....24
3.2.4 ACPI_INT_STATUS (0x00C)....24
3.2.5 PM1_CONTROL (0x010)....25
3.2.6 PM1_STATUS (0x014)....26
3.2.7 PM1_ENABLE (0x018)....27
3.2.8 PM_TIMER_VALUE (0x020)....27
3.2.9 PM_TIMER_CONFIG (0x024)....28
3.2.10 GPE0_STATUS_LO (0x030)....28
3.2.11 GPE0_STATUS_HI (0x034)....29
3.2.12 GPE0_ENABLE_LO (0x038)....29
3.2.13 GPE0_ENABLE_HI (0x03C)....29
3.2.14 CLOCK_GATE_CTRL (0x050)....30
3.2.15 CLOCK_GATE_STATUS (0x054)....30
3.2.16 POWER_DOMAIN_CTRL (0x058)....30
3.2.17 POWER_DOMAIN_STATUS (0x05C)....30
3.2.18 WAKE_STATUS (0x060)....31
3.2.19 WAKE_ENABLE (0x064)....31
3.2.20 RESET_CTRL (0x068)....32
3.2.21 RESET_STATUS (0x06C)....32
3.2.22 BUTTON_TIMING (0x070)....33
3.2.23 PM_TIMER_VALUE_HI (0x074)....34
3.2.24 PM_TIMER_MATCH (0x078)....34
3.2.25 PWR_SEQ_CONFIG (0x07C)....35
3.2.26 PWR_SEQ_STATUS (0x080)....36

3.2.27 GPE0_TRIGGER_LO / HI (0x084, 0x088) and GPE1_TRIGGER_LO / HI (0x0A4, 0x0A8)..... 36

3.2.28 GPE0_WAKE_EN_LO / HI (0x08C, 0x090) and GPE1_WAKE_EN_LO / HI (0x0AC, 0x0B0)..... 36

3.2.29 GPE1_STATUS_LO / HI (0x094, 0x098) and GPE1_ENABLE_LO / HI (0x09C, 0x0A0)..... 37

3.3 Design Notes..... 37

3.3.1 Where the status lives..... 37

3.3.2 What feeds pm_interrupt..... 37

3.3.3 Sleep and wake..... 38

3.3.4 S5, soft off..... 38

3.3.5 Storage-only fields..... 38

3.3.6 Deferred..... 39

3.4 Navigation..... 39

# List of Figures

Figure 1.1: PM/ACPI Block Diagram.....10

# List of Tables

*No tables in this document.*

# List of Waveforms

Waveform 1.1: Sleep Entry (S3 Suspend).....15
Waveform 1.2: Wake Event.....15
Waveform 1.3: PM Timer.....16
Waveform 1.4: General Purpose Event (GPE).....16

# 1 Pm Acpi Mas Index

**Generated:** 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 2 pm_acpi – Overview

## 2.1 Overview

The APB PM/ACPI controller provides ACPI-compatible power management functionality with an APB interface. It handles system power states, events, and timer functionality.

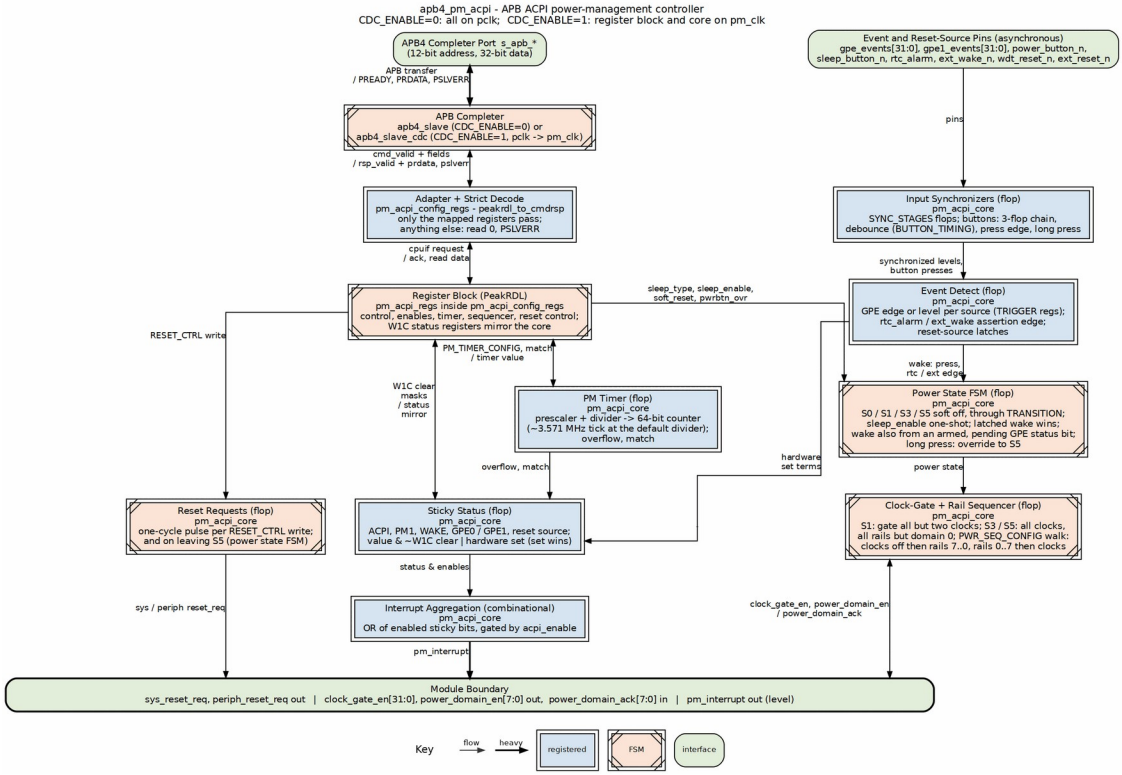
Key features:

- ACPI-style power management events
- Single PM1 control/status/enable block (no separate PM1a/PM1b)
- PM timer (32-bit, ~3.571 MHz at the default divider; 3.579545 MHz is the ACPI target – exact only with pm_clk = 100.227 MHz)
- GPE (General Purpose Events) support: 32 sources in one bank, rising-edge detected, per-bit sticky status with a W1C clear
- Clock gating control (32 domains) and power domain control (8 domains)
- System sleep state control (S0/S1/S3/S5)
- Single active-high `pm_interrupt` output (no separate SCI/SMI outputs), a level over the enabled sticky status bits
- Wake from GPE, power button, RTC alarm or an external pin, with the wake request latched so a one-cycle source lands in S0 and stays there

Applications:

- System power management
- Sleep state transitions (S0/S1/S3/S5)
- Wake event handling
- Power button events

### 2.1.1 Figure 1.1: PM/ACPI Block Diagram




*PM/ACPI Block Diagram*

## 2.2 Parameters

Top-level parameters on apb4_pm_acpi:

| Parameter | Description |
|---|---|
| `CDC_ENABLE` | 0 = single pclk domain, 1 = separate core clock via apb4_slave_cdc |
| `USE_JOHNSON` | CDC counter encoding (default 0) |
| `SYNC_STAGES` | Depth of the `rtc_alarm`, `ext_wake_n` and `gpe_events` input synchronizers in pm_acpi_core; must be at least 2 (default 2) |

There is no depth parameter – the CDC FIFO depth is hardcoded to 2 at the apb4_slave_cdc instantiation.

### 2.2.1 Input Synchronizers

Every device pin that feeds the core is asynchronous to it, in both CDC_ENABLE settings, so each one passes a synchronizer before anything looks at it. `rtc_alarm`, `ext_wake_n` and the 32 `gpe_events` bits go through SYNC_STAGES flops; `power_button_n` and `sleep_button_n` keep their own 3-flop chain, whose last two stages also supply the press edge detect. The cost is SYNC_STAGES clocks of latency on the first three (three clocks on the buttons), and a pulse shorter than one core clock may not be seen at all – drive any of them for at least two core-clock periods. The GPE edge reference tracks the synchronized input every cycle, so a level that changed while GPE was disabled does not fabricate an edge when software enables it. `rtc_alarm` and `ext_wake_n` are edge-latched the same way as GPE and the buttons: a rising-edge detector follows the synchronizer and the edge sets the sticky bit, so a held level sets it once, a W1C clears it while the pin is still asserted, and the pin must deassert and reassert to set it again.

One clocking caveat up front, because it bites: `pm_interrupt` and the clock-gate, power-domain and reset-request outputs are driven from the CORE clock domain (pm_clk when CDC_ENABLE=1) – synchronize them externally if you consume them on another clock.

## 2.3 Functional Description

### 2.3.1 Register Summary

Selected registers; see Chapter 5 for the complete map, fields, resets, and access types.

| Offset | Name | Access | Description |
|---|---|---|---|
| 0x000 | ACPI_CONTROL | RW | Global control and power state |
| 0x004 | ACPI_STATUS | W1C | Global status and power events |
| 0x008 | ACPI_INT_ENABLE | RW | Interrupt enable mask |
| 0x00C | ACPI_INT_STATUS | W1C | Interrupt status |
| 0x010 | PM1_CONTROL | RW | PM1 control (sleep, button override) |
| 0x014 | PM1_STATUS | W1C | PM1 status flags |
| 0x018 | PM1_ENABLE | RW | PM1 event enable mask |
| 0x020 | PM_TIMER_ | RO | PM Timer low word; |

| Offset | Name | Access | Description |
|---|---|---|---|
| | VALUE | | reading it snapshots the high word |
| 0x024 | PM_TIMER_CONFIG | RW | PM Timer divider, prescaler and 64-bit mode |
| 0x030 | GPE0_STATUS_LO | W1C | GPE status bits [15:0] |
| 0x034 | GPE0_STATUS_HI | W1C | GPE status bits [31:16] |
| 0x038 | GPE0_ENABLE_LO | RW | GPE enable bits [15:0] |
| 0x03C | GPE0_ENABLE_HI | RW | GPE enable bits [31:16] |
| 0x050 | CLOCK_GATE_CTRL | RW | Clock gating control [31:0] |
| 0x054 | CLOCK_GATE_STATUS | RO | Clock gate status |
| 0x058 | POWER_DOMAIN_CTRL | RW | Power domain control [7:0] |
| 0x05C | POWER_DOMAIN_STATUS | RO | Power domain status |
| 0x060 | WAKE_STATUS | W1C | Wake event sources |
| 0x064 | WAKE_ENABLE | RW | Wake event enable mask |
| 0x068 | RESET_CTRL | RW | Reset generation control |
| 0x06C | RESET_STATUS | RO | Reset source information |
| 0x070 | BUTTON_TIMING | RW | Button debounce window and long-press threshold |
| 0x074 | PM_TIMER_VALUE_HI | RO | PM Timer high word, snapshotted by reading the |

| Offset | Name | Access | Description |
|---|---|---|---|
| | | | low word |
| 0x078 | PM_TIMER_MATCH | RW | PM Timer comparator, on the low word |
| 0x07C | PWR_SEQ_CONFIG | RW | Rail sequencer enable, acknowledge requirement, gap |
| 0x080 | PWR_SEQ_STATUS | RO | Where the rail walk has got to |
| 0x084 | GPE0_TRIGGER_LO | RW | Edge or level per GPE0 source [15:0] |
| 0x088 | GPE0_TRIGGER_HI | RW | Edge or level per GPE0 source [31:16] |
| 0x08C | GPE0_WAKE_EN_LO | RW | GPE0 wake arming [15:0] |
| 0x090 | GPE0_WAKE_EN_HI | RW | GPE0 wake arming [31:16] |
| 0x094 | GPE1_STATUS_LO | W1C | GPE1 status bits [15:0] |
| 0x098 | GPE1_STATUS_HI | W1C | GPE1 status bits [31:16] |
| 0x09C | GPE1_ENABLE_LO | RW | GPE1 enable bits [15:0] |
| 0x0A0 | GPE1_ENABLE_HI | RW | GPE1 enable bits [31:16] |
| 0x0A4 | GPE1_TRIGGER_LO | RW | Edge or level per GPE1 source [15:0] |
| 0x0A8 | GPE1_TRIGGER_HI | RW | Edge or level per GPE1 source [31:16] |
| 0x0AC | GPE1_WAKE_EN_LO | RW | GPE1 wake arming [15:0] |
| 0x0B0 | GPE1_WAKE_EN_HI | RW | GPE1 wake arming [31:16] |

Only these thirty-eight addresses decode. Every other address in the 4 KB window is dropped – the write is ignored, the read returns 0 – and answered with PSLVERR.

### 2.3.2 Sticky Status and the Interrupt

Every W1C status bit in the map – ACPI_STATUS, ACPI_INT_STATUS, PM1_STATUS, WAKE_STATUS and GPE0_STATUS_LO/HI – lives in pm_acpi_core, and the register block only mirrors it. A hardware event sets the bit, and it holds until software writes a 1 to that bit position; a write of 0 does nothing, a write to one register never touches another, and a byte-strobed write (PSTRB) can only clear bits inside the enabled bytes. If a set and a clear land in the same cycle the set wins, so an event cannot fall between a read and its acknowledge.

`pm_interrupt` is a level: the OR of the enabled sticky bits, deasserting only once software has cleared every enabled source. Three groups feed it:

| Term | Status bit | Enabled by |
|---|---|---|
| PME, wake, timer overflow, state transition | ACPI_STATUS bit N | ACPI_INT_ENABLE bit N |
| PM1 timer, power button, sleep button, RTC | PM1_STATUS bit N | PM1_ENABLE bit N, then ACPI_INT_ENABLE.pm1_enable |
| GPE source N | GPE0_STATUS bit N | GPE0_ENABLE bit N, then ACPI_INT_ENABLE.gpe_int_enable |

PM1_STATUS.wak_sts has no enable bit, matching ACPI: a wake is reported, it is not an interrupt source on its own, so it is masked out of the PM1 term. ACPI_CONTROL.acpi_enable is the SCI_EN-like master gate: it gates GPE event capture and the pin itself, while PM1_STATUS and WAKE_STATUS keep recording their events with it clear, so software can see what happened before it enabled the block. ACPI_INT_STATUS is an unconditional per-source event log of the same events: each bit sets when its event occurs whether or not the matching enable is set (gpe_int from the GPE edge events, not the pending level), each clears by its own W1C on its own, and none of it feeds the pin – if it did, dropping one interrupt would mean clearing two registers.

### 2.3.3 Power State FSM

Five states: S0 working, S1 sleep, S3 suspend, S5 soft off and a one-cycle TRANSITION state the machine passes through on the way in and on the way out. PM1_CONTROL.sleep_enable is a one-shot request (it self-clears and the core edge-detects it), so one write is one sleep request: from S0, a request with sleep_type 1, 3 or 5 goes through TRANSITION into S1, S3 or S5; any other

sleep_type stays in S0. From S1, S3 or S5, any enabled wake source goes through TRANSITION back to S0 (leaving S5 also pulses `sys_reset_req`).

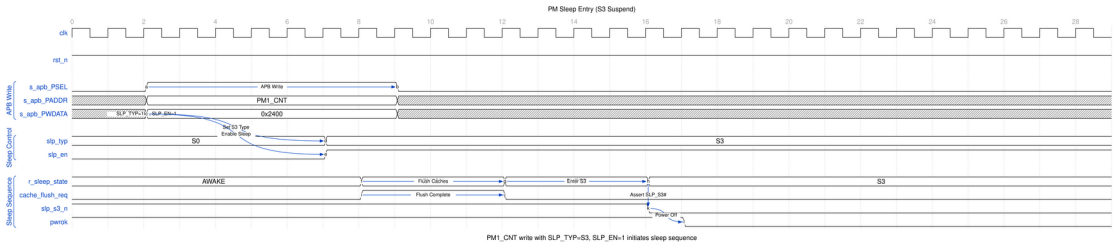
The part that matters for a pulsed source: the core latches the wake request while it is out of S0, and in TRANSITION a latched or live wake outranks the still-programmed sleep_type. A one-cycle power-button press therefore lands in S0 and stays there, and software does not have to unprogram sleep_type after a wake. The latch drops on reaching S0 and on the next sleep request. A wake event that lands in the exact cycle of the sleep request is not latched as a wake: the sticky status bit and the level interrupt still record it, the machine enters the programmed sleep state, and the next wake event brings it back. ACPI_CONTROL.soft_reset (one core-clock pulse per write, like the RESET_CTRL requests) forces the machine back to S0, clears every sticky status register and the latched wake, and leaves configuration alone.

## 2.4 Waveforms

> Note: The rendered waveforms use illustrative, ACPI-generic signal names (for example SLP_S3#, SCI#) that do not all correspond to RTL ports. The RTL has no dedicated sleep-state pins or cache-flush request, and its only interrupt is the single active-high `pm_interrupt`. Refer to the register map (Chapter 5) for the authoritative register and field names.

### 2.4.1 Waveform 1.1: Sleep Entry (S3 Suspend)

Software initiates sleep by writing PM1_CONTROL (0x010).

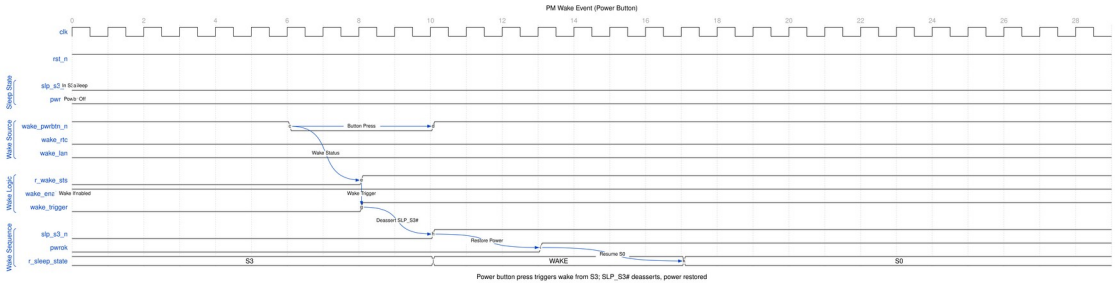



*PM Sleep Entry*

The sequence: 1. Software writes PM1_CONTROL with sleep_type (0=S0, 1=S1, 3=S3) and sleep_enable 2. The PM core FSM enters the requested sleep state (S1 or S3) 3. Clock gating / power domain outputs are updated for the target state 4. On completion, ACPI_STATUS.state_transition sets and holds until software writes 1 to clear it; `pm_interrupt` asserts if ACPI_INT_ENABLE.state_trans_enable is set

### 2.4.2 Waveform 1.2: Wake Event

A wake source triggers a return to S0 from sleep.



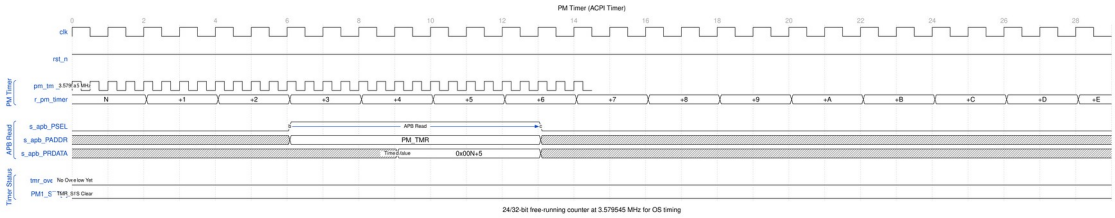

## *PM Wake Event*

Wake sequence: 1. Enabled wake source detected (power button press, RTC alarm, external pin or an enabled GPE), after its synchronizer 2. Wake status set in WAKE_STATUS and PM1_STATUS.wak_sts, and the wake request latched in the core 3. The FSM passes through TRANSITION back to S0; the latched request outranks the still-programmed sleep_type, so a one-cycle source such as the button stays in S0 4. `pm_interrupt` asserts if the corresponding enable is set, and holds until software clears the status

The waveform is drawn with the power button, the one source that is a single-cycle pulse; the level-driven sources (RTC alarm, `ext_wake_n`, GPE) are edge-latched after their synchronizers and follow the same path.

### 2.4.3 Waveform 1.3: PM Timer

Free-running PM timer for timing services.

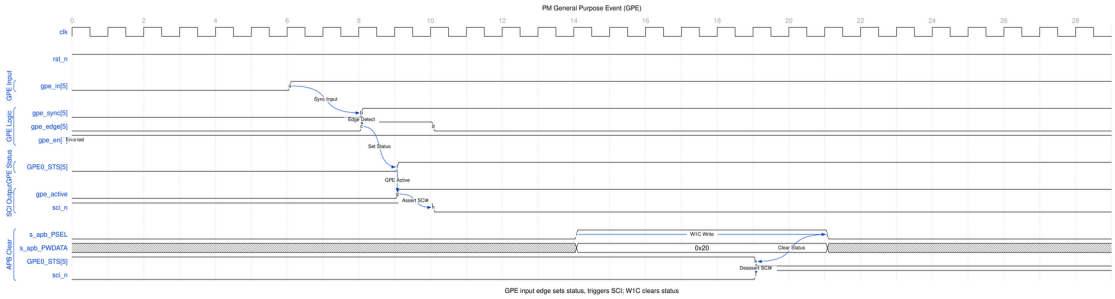



## *PM Timer*

The 32-bit free-running counter (PM_TIMER_VALUE, 0x020) increments at ~3.571 MHz from a 100 MHz pm_clk using the PM_TIMER_CONFIG divider (default 0x001B, divide-by-28; 0.23% below the ACPI-standard 3.579545 MHz). Overflow sets ACPI_STATUS.timer_overflow and PM1_STATUS.tmr_sts, both sticky until cleared, and `pm_interrupt` holds while either bit is set and enabled.

### 2.4.4 Waveform 1.4: General Purpose Event (GPE)

External events set a GPE status bit and can raise `pm_interrupt`.




*PM GPE Event*

A rising edge on a synchronized GPE input sets that one bit in GPE0_STATUS_LO/HI (when ACPI and GPE are enabled) and it holds. If the source is enabled in GPE0_ENABLE and ACPI_INT_ENABLE.gpe_int_enable is set, `pm_interrupt` asserts; software reads the status, services the event and writes 1 to the bit, which clears it, drops the interrupt and – because the same pending term feeds the GPE wake – unblocks the next sleep entry. Clearing is never gated by the enables, so software can always drain the register.

---

## 2.5 Navigation

**Next:** Chapter 2 (Architecture) is planned and not yet written – see the index

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 3 pm_acpi – Register Map

## 3.1 Overview

This register map is generated from the PeakRDL specification (`rdl/pm_acpi/pm_acpi_regs.rdl`) and matches the synthesized register block in `rtl/pm_acpi/pm_acpi_regs.sv`. Offsets are byte offsets from the block base address. All registers are 32 bits wide with 32-bit access.

> Note: This block does not implement the classic ACPI PM1a/PM1b fixed-hardware layout (PM1_STS/PM1_EN/PM1_CNT at 0x00-0x0C, GPE0/GPE1). It is a custom PeakRDL register file. Fields such as BM_STS, GBL_STS, GBL_EN, SCI_EN, BM_RLD, GBL_RLS and a second GPE bank (GPE1) do not exist in the RTL.

### 3.1.1 Register Summary

| Offset | Name | Access | Reset | Description |
|---|---|---|---|---|
| 0x000 | ACPI_CONTROL | RW | 0x00000000 | Global control and power state |
| 0x004 | ACPI_STATUS | W1C | 0x00000000 | Global status and power events |
| 0x008 | ACPI_INT_ENABLE | RW | 0x00000000 | Interrupt enable mask |
| 0x00C | ACPI_INT_STATUS | W1C | 0x00000000 | Interrupt status |
| 0x010 | PM1_CONTROL | RW | 0x00000000 | PM1 control (sleep, button override) |
| 0x014 | PM1_STATUS | W1C | 0x00000000 | PM1 status flags |
| 0x018 | PM1_ENABLE | RW | 0x00000000 | PM1 event enable mask |
| 0x01C | (Reserved) | - | - | Not decoded: dropped with PSLVERR |
| 0x020 | PM_TIMER_VALUE | RO | 0x00000000 | PM Timer current value (32-bit) |
| 0x024 | PM_TIMER_CONFI | RW | 0x0000001B | PM Timer clock divider |

| Offset | Name | Access | Reset | Description |
|---|---|---|---|---|
| | G | | | |
| 0x028-0x02C | (Reserved) | - | - | Not decoded: dropped with PSLVERR |
| 0x030 | GPE0_STATUS_LO | W1C | 0x00000000 | GPE0 status bits [15:0] |
| 0x034 | GPE0_STATUS_HI | W1C | 0x00000000 | GPE0 status bits [31:16] |
| 0x038 | GPE0_ENABLE_LO | RW | 0x00000000 | GPE0 enable bits [15:0] |
| 0x03C | GPE0_ENABLE_HI | RW | 0x00000000 | GPE0 enable bits [31:16] |
| 0x040-0x04C | (Reserved) | - | - | Not decoded: dropped with PSLVERR |
| 0x050 | CLOCK_GATE_CTRL | RW | 0xFFFFFFFF | Clock gating control [31:0] |
| 0x054 | CLOCK_GATE_STATUS | RO | 0xFFFFFFFF | Clock gate status (reads the live core state; all gates enabled at reset) |
| 0x058 | POWER_DOMAIN_CTRL | RW | 0x000000FF | Power domain control [7:0] |
| 0x05C | POWER_DOMAIN_STATUS | RO | 0x000000FF | Power domain status (reads the live core state; all domains powered at reset) |
| 0x060 | WAKE_STATUS | W1C | 0x00000000 | Wake event sources |
| 0x064 | WAKE_ENABLE | RW | 0x00000000 | Wake event enable mask |
| 0x068 | RESET_CTRL | RW | 0x00000000 | Reset generation control |
| 0x06C | RESET_STATUS | RO | 0x00000001 | Reset source information (por_reset |

| Offset | Name | Access | Reset | Description |
|---|---|---|---|---|
| | | | | reads 1 out of reset) |
| 0x070 | BUTTON_TIMING | RW | 0x1C000000 | Button debounce window and long-press threshold |
| 0x074 | PM_TIMER_VALUE_HI | RO | 0x00000000 | High word of the PM timer, as snapshotted by the last read of PM_TIMER_VALUE |
| 0x078 | PM_TIMER_MATCH | RW | 0x00000000 | PM timer comparator, on the low word |
| 0x07C | PWR_SEQ_CONFIG | RW | 0x00000000 | Rail sequencer: enable, acknowledge requirement, inter-rail gap |
| 0x080 | PWR_SEQ_STATUS | RO | 0x00000000 | Where the rail walk has got to |
| 0x084 | GPE0_TRIGGER_LO | RW | 0x00000000 | Edge or level per GPE0 source [15:0] |
| 0x088 | GPE0_TRIGGER_HI | RW | 0x00000000 | Edge or level per GPE0 source [31:16] |
| 0x08C | GPE0_WAKE_EN_LO | RW | 0x00000000 | GPE0 wake arming [15:0] |
| 0x090 | GPE0_WAKE_EN_HI | RW | 0x00000000 | GPE0 wake arming [31:16] |
| 0x094 | GPE1_STATUS_LO | W1C | 0x00000000 | GPE1 status [15:0] |
| 0x098 | GPE1_STATUS_HI | W1C | 0x00000000 | GPE1 status [31:16] |
| 0x09C | GPE1_ENABLE_LO | RW | 0x00000000 | GPE1 enable [15:0] |

| Offset | Name | Access | Reset | Description |
|---|---|---|---|---|
| 0x0A0 | GPE1_ENABLE_HI | RW | 0x00000000 | GPE1 enable [31:16] |
| 0x0A4 | GPE1_TRIGGER_LO | RW | 0x00000000 | Edge or level per GPE1 source [15:0] |
| 0x0A8 | GPE1_TRIGGER_HI | RW | 0x00000000 | Edge or level per GPE1 source [31:16] |
| 0x0AC | GPE1_WAKE_EN_LO | RW | 0x00000000 | GPE1 wake arming [15:0] |
| 0x0B0 | GPE1_WAKE_EN_HI | RW | 0x00000000 | GPE1 wake arming [31:16] |
| 0x0B4-0xFFC | (Reserved) | - | - | Not decoded: dropped with PSLVERR |

Access legend: RW = read/write, RO = read-only (hardware-updated), W1C = read status / write 1 to clear.

The decode is strict. Only the thirty-eight mapped registers are visible to software, compared on the whole 12-bit address, register by register. Everything else in the 4 KB window – the gaps at 0x01C, 0x028-0x02C and 0x040-0x04C as much as the space above 0x0B0 – is dropped: the write is ignored, the read returns 0, and the access is answered with PSLVERR. There is no aliasing anywhere in the window (the map used to repeat every 0x80 bytes because only PADDR[6:0] reached the register block; fixed 2026-09-09, issue #54). The register block decodes eight address bits now that PWR_SEQ_STATUS sits at 0x080, so the first alias the strict decode rejects is 0x100 rather than 0x080.

Every W1C status register in this map is a live mirror of a register that pm_acpi_core owns. The hardware event sets a bit and it holds until software writes a 1 to that bit; a write of 0 is a no-op, a write to one status register never touches another, PSTRB is honoured so a byte-strobed write can only clear bits inside the enabled bytes, and a set that lands in the same cycle as a clear wins. ACPI_CONTROL.soft_reset clears all of them at once.

---

## 3.2 Functional Description

Bit-level definitions, one section per register.

### 3.2.1 ACPI_CONTROL (0x000)

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 0 | acpi_enable | RW | 0 | SCI_EN-like master enable. Gates GPE event capture (no GPE0_STATUS bit sets while it is 0) and the `pm_interrupt` pin (held low while it is 0). PM1_STATUS and WAKE_STATUS still record their events while it is 0, so software can see what happened before it enabled the block |
| 1 | pm_timer_enable | RW | 0 | Enable PM Timer (0=stopped, 1=running) |
| 2 | gpe_enable | RW | 0 | Enable GPE event processing |
| 5:4 | current_state | RO | 0 | Current power state (0=S0, 1=S1, 3=S3), hardware-updated; reads 0 (S0) while the FSM is in its transition state |
| 6 | low_power_req | RW | 0 | Storage only, no hardware effect. There is no low-power mode distinct from S1/S3; sleep entry goes through PM1_CONTROL. Reads back what was written |
| 7 | soft_reset | RW | 0 | Write 1 to soft-reset the PM controller (self-clearing, always reads |

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| | | | | 0). Exactly one core-clock pulse per write, however many cycles the bus holds the write – the wrapper edge-detects the field. Clears every sticky status register (ACPI_STATUS, ACPI_INT_STATUS, PM1_STATUS, WAKE_STATUS, GPE0_STATUS_LO/HI), drops the latched wake request, returns the FSM to S0 and switches RESET_STATUS from por_reset to sw_reset. Configuration registers are untouched |
| 31:8 | reserved | RO | 0 | Reserved |

### 3.2.2 ACPI_STATUS (0x004)

Write 1 to clear each bit. Each bit, ANDed with its ACPI_INT_ENABLE bit, is one term of `pm_interrupt`.

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 0 | pme_status | W1C | 0 | Power Management Event occurred: a button press, an enabled wake event or a completed state transition |
| 1 | wake_status | W1C | 0 | An enabled wake event occurred |
| 2 | timer_ove | W1C | 0 | PM Timer overflow |

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| | rflow | | | occurred |
| 3 | state_transition | W1C | 0 | Power state transition complete (the cycle the FSM leaves TRANSITION) |
| 31:4 | reserved | RO | 0 | Reserved |

### 3.2.3 ACPI_INT_ENABLE (0x008)

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 0 | pme_enable | RW | 0 | Enable interrupt on PME event |
| 1 | wake_enable | RW | 0 | Enable interrupt on wake event |
| 2 | timer_ovf_enable | RW | 0 | Enable interrupt on PM timer overflow |
| 3 | state_trans_enable | RW | 0 | Enable interrupt on power state transition |
| 4 | pm1_enable | RW | 0 | Enable interrupt on any PM1 event |
| 5 | gpe_int_enable | RW | 0 | Enable interrupt on any GPE event |
| 31:6 | reserved | RO | 0 | Reserved |

### 3.2.4 ACPI_INT_STATUS (0x00C)

Write 1 to clear each bit. This register is an unconditional per-source event log: each bit sets when its event occurs, whether or not the corresponding enable is set, and each bit is cleared by its own W1C independently of ACPI_STATUS, PM1_STATUS and GPE0_STATUS. It does not feed `pm_interrupt` (the enabled bits of those three do), so dropping an interrupt means clearing one register, not two, and this one can be cleared before or after the register that owns the interrupt.

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 0 | pme_int | W1C | 0 | PME event recorded |
| 1 | wake_int | W1C | 0 | Wake event recorded |

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 2 | timer_ovf_int | W1C | 0 | Timer overflow event recorded |
| 3 | state_trans_int | W1C | 0 | State transition event recorded |
| 4 | pm1_int | W1C | 0 | PM1 event recorded: set whenever any PM1_STATUS bit sets (timer, power button, sleep button, RTC alarm, wake), enabled or not |
| 5 | gpe_int | W1C | 0 | GPE event recorded: set from the GPE edge events, not from the GPE pending level, so it can be cleared before or after GPE0_STATUS |
| 31:6 | reserved | RO | 0 | Reserved |

### 3.2.5 PM1_CONTROL (0x010)

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 2:0 | sleep_type | RW | 0 | Sleep type (0=S0, 1=S1, 3=S3); other values are treated as 0 and the FSM stays in S0 |
| 3 | sleep_enable | RW | 0 | Write 1 to request entry to the state in sleep_type. One-shot: self-clearing (always reads 0) and rising-edge detected in the core, so one write is one sleep request and a wake that returns the machine to S0 is not undone by the |

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| | | | | still-programmed sleep_type |
| 4 | pwrbtn_ovr | RW | 0 | Storage only, no hardware effect. The power button always sets PM1_STATUS.pwrbtn_sts; mask its interrupt with PM1_ENABLE.pwrbtn_en and its wake with WAKE_ENABLE.pwrbtn_wake_en |
| 5 | slpbtn_ovr | RW | 0 | Storage only, no hardware effect. Mask the sleep button's interrupt with PM1_ENABLE.slpbtn_en |
| 31:6 | reserved | RO | 0 | Reserved |

### 3.2.6 PM1_STATUS (0x014)

Write 1 to clear each bit. The event is recorded whether or not its PM1_ENABLE bit is set, and whether or not ACPI_CONTROL.acpi_enable is set; the enables only gate the interrupt. rtc_sts is edge-latched: `rtc_alarm` passes the synchronizer and then a rising-edge detector, and the edge sets the bit. A W1C clears it even while the alarm is still asserted, and the alarm must deassert and reassert to set it again – the same rule as the GPE bits and the buttons.

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 0 | tmr_sts | W1C | 0 | PM Timer carry/overflow |
| 1 | pwrbtn_sts | W1C | 0 | Power button pressed |
| 2 | slpbtn_sts | W1C | 0 | Sleep button pressed |
| 3 | rtc_sts | W1C | 0 | RTC alarm occurred |
| 4 | wak_sts | W1C | 0 | System wake event. Has |

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| | | | | no PM1_ENABLE bit (ACPI reports a wake, it is not an interrupt source on its own), so it never contributes to `pm_interrupt` |
| 31:5 | reserved | RO | 0 | Reserved |

### 3.2.7 PM1_ENABLE (0x018)

Each bit gates its source's contribution to `pm_interrupt` (together with ACPI_INT_ENABLE.pm1_enable); PM1_STATUS still records the event either way.

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 0 | tmr_en | RW | 0 | Enable PM timer events |
| 1 | pwrbtn_en | RW | 0 | Enable power button events |
| 2 | slpbtn_en | RW | 0 | Enable sleep button events |
| 3 | rtc_en | RW | 0 | Enable RTC alarm events |
| 31:4 | reserved | RO | 0 | Reserved |

### 3.2.8 PM_TIMER_VALUE (0x020)

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 31:0 | timer_value | RO | 0 | Low 32 bits of the PM timer count (hardware-updated). Reading this |

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| | | | | SNAPSHOTS the high word into PM_TIMER_VALUE_HI, so a pair of reads cannot straddle a carry |

The counter is 32 bits and increments at a divided clock rate (see PM_TIMER_CONFIG). At the default divider it advances at ~3.571 MHz from a 100 MHz pm_clk (100/28 – 0.23% below the ACPI-standard 3.579545 MHz; an exact tick needs pm_clk = 100.227 MHz) and rolls over roughly every 1200 seconds.

---

### 3.2.9 PM_TIMER_CONFIG (0x024)

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 15:0 | timer_div | RW | 0x001B | Clock divider: timer_clk = pm_clk / (timer_div + 1) |
| 31:16 | reserved | RO | 0 | Reserved |

Reset value 0x001B (27 decimal) divides by 28, yielding a 3.5714 MHz tick from a 100.000 MHz pm_clk – 0.23% below the ACPI-standard 3.579545 MHz (an exact tick needs pm_clk = 100.227 MHz or a fractional divider).

---

### 3.2.10 GPE0_STATUS_LO (0x030)

Covers GPE sources 0-15. A bit sets on a rising edge of its synchronized `gpe_events` input while ACPI_CONTROL.acpi_enable and gpe_enable are both set, and holds until software writes a 1 to it. Clearing is never gated by the enables, so the register can always be drained. Each bit ANDed with its GPE0_ENABLE bit is a pending GPE; the OR of those, gated by ACPI_INT_ENABLE.gpe_int_enable, is the GPE term of `pm_interrupt`, and the same OR gated by WAKE_ENABLE.gpe_wake_en is the GPE wake. Clearing the bit therefore drops the interrupt and unblocks the next sleep entry.

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 15:0 | gpe_status | W1C | 0 | General Purpose Event status bits |

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| | | | | 0-15 |
| 31:16 | reserved | RO | 0 | Reserved |

### 3.2.11 GPE0_STATUS_HI (0x034)

Covers GPE sources 16-31, with the same set, hold and clear rules as GPE0_STATUS_LO.

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 15:0 | gpe_status | W1C | 0 | General Purpose Event status bits 16-31 |
| 31:16 | reserved | RO | 0 | Reserved |

### 3.2.12 GPE0_ENABLE_LO (0x038)

Covers GPE sources 0-15.

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 15:0 | gpe_enable | RW | 0 | General Purpose Event enable bits 0-15 |
| 31:16 | reserved | RO | 0 | Reserved |

### 3.2.13 GPE0_ENABLE_HI (0x03C)

Covers GPE sources 16-31.

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 15:0 | gpe_enable | RW | 0 | General Purpose Event enable bits 16-31 |
| 31:16 | reserved | RO | 0 | Reserved |

The 32 GPE sources are exposed as two 16-bit LO/HI register pairs. There is no second GPE bank (no GPE1).

---

### 3.2.14 CLOCK_GATE_CTRL (0x050)

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 31:0 | clk_gate_ctrl | RW | 0xFFFFFFFF | Clock gate enable per block (0=gated/off, 1=enabled/on) |

### 3.2.15 CLOCK_GATE_STATUS (0x054)

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 31:0 | clk_gate_status | RO | 0xFFFFFFFF (live) | Actual clock gate state per block (reads the core register, which resets all-enabled) |

### 3.2.16 POWER_DOMAIN_CTRL (0x058)

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 7:0 | pwr_domain_ctrl | RW | 0xFF | Power domain enable per domain (0=off, 1=on) |
| 31:8 | reserved | RO | 0 | Reserved |

### 3.2.17 POWER_DOMAIN_STATUS (0x05C)

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 7:0 | pwr_domain_status | RO | 0xFF (live) | Actual power domain state (reads the core register, which resets all-powered) |
| 31:8 | reserved | RO | 0 | Reserved |

Sleep states override the CTRL values: in S1 the core forces clock_gate_en = CLOCK_GATE_CTRL & 0x3 (only blocks 0-1 may stay on); in S3 clock_gate_en = 0 and power_domain_en = POWER_DOMAIN_CTRL & 0x1 (domain 0 always-on). The STATUS registers therefore read back values software never wrote while sleeping.

### 3.2.18 WAKE_STATUS (0x060)

Write 1 to clear each bit. A bit sets when its source fires while the matching WAKE_ENABLE bit is set, whatever ACPI_CONTROL.acpi_enable holds; the same event sets PM1_STATUS.wak_sts and ACPI_STATUS.wake_status and arms the core's latched wake request. The RTC and external sources are edge-latched: `rtc_alarm` and `ext_wake_n` pass their synchronizers and then a rising-edge detector, and the edge sets the bit, so a W1C clears it while the pin is still asserted and the pin must deassert and reassert to set it again – the same rule as GPE and the buttons. The GPE wake request itself stays pending until its GPE0_STATUS bit is cleared.

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 0 | gpe_wake | W1C | 0 | Woke from GPE event |
| 1 | pwrbtn_wake | W1C | 0 | Woke from power button |
| 2 | rtc_wake | W1C | 0 | Woke from RTC alarm |
| 3 | ext_wake | W1C | 0 | Woke from external signal |
| 31:4 | reserved | RO | 0 | Reserved |

### 3.2.19 WAKE_ENABLE (0x064)

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 0 | gpe_wake_en | RW | 0 | Enable wake from GPE events |
| 1 | pwrbtn_wake_en | RW | 0 | Enable wake from power button |
| 2 | rtc_wake_en | RW | 0 | Enable wake from RTC alarm |

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 3 | ext_wake_en | RW | 0 | Enable wake from external signal |
| 31:4 | reserved | RO | 0 | Reserved |

### 3.2.20 RESET_CTRL (0x068)

Both request fields are one-shots. A write of 1 produces exactly one core-clock pulse on the output, however many cycles the bus holds the write, because the wrapper edge-detects the singlepulse field; the same rule applies to ACPI_CONTROL.soft_reset.

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 0 | sys_reset | RW | 0 | Write 1 to pulse the `sys_reset_req` output for exactly one core-clock cycle per write (self-clearing, always reads 0). What the system does with the request is outside this block |
| 1 | periph_reset | RW | 0 | Write 1 to pulse the `periph_reset_req` output for exactly one core-clock cycle per write (self-clearing, always reads 0) |
| 31:2 | reserved | RO | 0 | Reserved |

### 3.2.21 RESET_STATUS (0x06C)

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 0 | por_reset | RO | 1 | Sticky level: reads 1 from hardware reset until |

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| | | | | ACPI_CONTROL.soft_reset executes, which hands the last-reset title to sw_reset |
| 1 | wdt_reset | RO | 0 | Sticky: LATCHED from the `wdt_reset_n` pin, because the pulse that caused a reset is long gone by the time software reads this. Tie the pin high if the system has no watchdog |
| 2 | sw_reset | RO | 0 | Sticky level: reads 1 once ACPI_CONTROL.soft_reset has executed since the last hardware reset (por_reset drops in the same cycle) |
| 3 | ext_reset | RO | 0 | Sticky: LATCHED from the `ext_reset_n` pin, same argument as wdt_reset. Tie the pin high if the system has no external reset button |
| 31:4 | reserved | RO | 0 | Reserved |

### 3.2.22 BUTTON_TIMING (0x070)

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 23:0 | debounce_cycles | RW | 0 | A candidate button level has to hold for this many core clocks before it is accepted; any change inside the window restarts the |

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| | | | | count. 0 accepts immediately, which is the behaviour the block had before the debouncer existed |
| 28:24 | long_press_shift | RW | 28 | Holding the ACCEPTED power-button level for 2^this cycles is ACPI's power-button override. 0 disables it |
| 31:29 | reserved | RO | 0 | Reserved |

A synchronizer resolves metastability and does nothing about contact bounce, which is why one press used to be recorded as several. **A press has to survive the debounce window to be seen at all**, so a test or a driver that programs a window must put the reset value back before anything else relies on a short press.

The long-press override is ENABLED by `PM1_CONTROL.pwrbtn_ovr`, not commanded by it.

---

### 3.2.23 PM_TIMER_VALUE_HI (0x074)

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 31:0 | value_hi | RO | 0 | Bits [63:32] of the counter, as latched by the last read of PM_TIMER_VALUE |

**Read the low word first.** This is a SNAPSHOT, not a live view, so the two halves always belong to the same instant. Reading the high word alone returns a stale snapshot, which is the point: a pair of live reads either side of a carry would return a value the counter never held.

---

### 3.2.24 PM_TIMER_MATCH (0x078)

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 31:0 | match_value | RW | 0 | Compare value for the PM timer's LOW word |

When the counter reaches it, `ACPI_STATUS.timer_match` sets and, with `ACPI_INT_ENABLE.timer_match_enable`, the interrupt asserts. The comparison is on the low 32 bits whatever `PM_TIMER_CONFIG.timer_64bit` says, so in 64-bit mode it recurs once per wrap of the low word. The match fires on the value the counter is ABOUT to hold, so the event and the value software can read agree.

---

### 3.2.25 PWR_SEQ_CONFIG (0x07C)

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 0 | seq_enable | RW | 0 | 0 = instant transitions, 1 = walk the rails |
| 1 | seq_ack_enable | RW | 0 | Each rail step waits for the matching `power_domain_ack` bit to report the commanded level |
| 15:2 | reserved | RO | 0 | Reserved |
| 31:16 | seq_delay | RW | 0 | Core clocks between one rail step and the next, and between the clock step and the first rail step |

Clock-gate and rail transitions are INSTANT unless this is enabled: every rail moves in the same cycle and the clocks move with them. That is fine in simulation and wrong on a board, where rail ordering is a correctness property rather than a performance one.

| direction | order |
|---|---|
| powering down | gate the clocks, wait, then rail 7 down to rail 0 |
| powering up | rail 0 up to rail 7, wait, then ungate the clocks |

So a domain is never clocked while its rail is down, and rails leave in the reverse of the order they arrived. A rail that never acknowledges STALLS THE WALK, which is the honest outcome: the rail did not come up. There is no timeout, because a made-up one would turn a board fault into a silent half-powered state.

---

### 3.2.26 PWR_SEQ_STATUS (0x080)

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 0 | seq_busy | RO | 0 | A rail walk is in progress |
| 3:1 | reserved | RO | 0 | Reserved |
| 6:4 | seq_index | RO | 0 | The rail the walk is currently on |
| 7 | reserved | RO | 0 | Reserved |
| 8 | seq_dir | RO | 0 | 1 = powering down (walking 7 to 0), 0 = powering up |
| 31:9 | reserved | RO | 0 | Reserved |

Read this when a power state change does not complete: `seq_busy` stuck with `seq_index` parked on one rail means that rail has not acknowledged.

---

### 3.2.27 GPE0_TRIGGER_LO / HI (0x084, 0x088) and GPE1_TRIGGER_LO / HI (0x0A4, 0x0A8)

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 15:0 | gpe_trigger | RW | 0 | 0 = edge, 1 = level, per source |
| 31:16 | reserved | RO | 0 | Reserved |

An EDGE source sets its status bit once, on the rising edge, and the bit then belongs to software: a source that is still asserted does not set it again. A LEVEL source sets its bit for as long as it is asserted, so a W1C while the source is still high has no lasting effect. That difference is the whole point – it is how software tells an event it missed from one that is still happening.

---

### 3.2.28 GPE0_WAKE_EN_LO / HI (0x08C, 0x090) and GPE1_WAKE_EN_LO / HI (0x0AC, 0x0B0)

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 15:0 | gpe_wake_enable | RW | 0 | Arms this source as a |

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| | | | | WAKE reason |
| 31:16 | reserved | RO | 0 | Reserved |

Only consulted when `ACPI_CONTROL.gpe_split_enable` is set. With the split off, GPEx_ENABLE arms both the interrupt and the wake and these registers are ignored.

| split | GPEx_ENABLE arms | GPEx_WAKE_EN arms |
|---|---|---|
| 0 (reset) | the interrupt and the wake | nothing |
| 1 | the interrupt only | the wake only |

With the split on, a source can wake a sleeping machine without interrupting a running one, which is what ACPI wants for a device the OS drives directly while awake.

---

### 3.2.29 GPE1_STATUS_LO / HI (0x094, 0x098) and GPE1_ENABLE_LO / HI (0x09C, 0x0A0)

The second ACPI GPE block, on the `gpe1_events` pins. Field layout, W1C behaviour and enable semantics are identical to GPE0; the two banks share the GPE interrupt and wake terms, so software that uses only one bank never sees the other.

---

## 3.3 Design Notes

### 3.3.1 Where the status lives

The register block stores configuration and mirrors status. Every W1C bit – ACPI_STATUS, ACPI_INT_STATUS, PM1_STATUS, WAKE_STATUS and GPE0_STATUS_LO/HI – is a flop in pm_acpi_core of the shape `(status & ~software_clear) | hardware_set`, and pm_acpi_config_regs turns a software write into a one-cycle per-bit clear mask (write data ANDed with the byte enables, at that register's own address). That is why a set wins over a same-cycle clear, why a write to one register cannot disturb another, and why PSTRB is honoured. This replaced a register block that held the state itself with an undriven update input, so that single-bit status could not hold and a GPE edge set all sixteen bits at once (fixed 2026-09-09, issue #54).

### 3.3.2 What feeds pm_interrupt

`pm_interrupt` is a level, the OR of these enabled sticky bits, and it deasserts only when software has cleared every enabled source:

| Term | Status | Enable |
|---|---|---|
| PME, wake, timer overflow, state transition | ACPI_STATUS bit N | ACPI_INT_ENABLE bit N |
| PM1 | PM1_STATUS bit N (wak_sts excluded) | PM1_ENABLE bit N, then ACPI_INT_ENABLE.pm1_enable |
| GPE | GPE0_STATUS bit N | GPE0_ENABLE bit N, then ACPI_INT_ENABLE.gpe_int_enable |

ACPI_CONTROL.acpi_enable sits in front of the pin, SCI_EN-like: while it is 0 `pm_interrupt` stays low and no GPE event is captured, but PM1_STATUS and WAKE_STATUS keep recording their events, so software can see what happened before it enabled the block.

ACPI_INT_STATUS is not in that OR. It is an unconditional per-source event log: each bit sets when its event occurs whether or not the corresponding enable is set (gpe_int from the GPE edge events, not from the GPE pending level), and each bit is cleared by its own W1C independently, so it can be cleared before or after the register that feeds the pin.

### 3.3.3 Sleep and wake

PM1_CONTROL.sleep_enable is a one-shot: from S0, one write with sleep_type 1, 3 or 5 takes the FSM through TRANSITION into S1, S3 or S5. In S1 or S3 any enabled wake source takes it through TRANSITION back to S0, and the core latches the wake request so that in TRANSITION a latched or live wake outranks the still-programmed sleep_type. A one-cycle power-button press lands in S0 and stays there; software does not have to unprogram sleep_type after a wake. The latch drops on reaching S0 and on the next sleep request. One corner: a wake event that lands in the exact cycle of the sleep request is not latched as a wake – the sticky status bit and the level interrupt still record it, the machine enters the programmed sleep state, and the next wake event brings it back.

### 3.3.4 S5, soft off

S5 is as dark as S3 – every clock gated, every domain but the always-on one powered down – but it retains nothing, so LEAVING it pulses `sys_reset_req`. A wake from soft off is a boot, not a resume. `current_state` is two bits, so it reports an ENCODING rather than the ACPI number: 0 = S0, 1 = S1, 2 = S5, 3 = S3. Encoding 2 was the free one, and widening the field would have moved bits software already reads.

### 3.3.5 Storage-only fields

ACPI_CONTROL.low_power_req and PM1_CONTROL.slpbtn_ovr read and write but are not routed to the core; there is no low-power mode distinct from S1/S3/S5, and “override the sleep button” never named a concrete effect. They are documented as storage, not as defects.

PM1_CONTROL.pwrbtn_ovr is no longer storage: it ENABLES the hardware long-press override rather than commanding it. Commanding soft off from a control bit would mean any write that happens to set the bit parks the machine in S5, which is not what a register called “override” should do to a register sweep.

### 3.3.6 Deferred

Legacy replacement routing (IRQ0 timer, IRQ8 RTC) and processor C/P-state hints are out of scope rather than deferred. RLB TASK-009 (was RLB-009), the deferred-feature list, is otherwise closed.

---

## 3.4 Navigation

**Back to:** PM/ACPI Specification Index